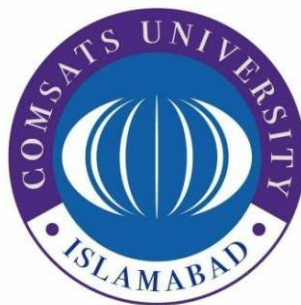


COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course</i>	<i>Info. Security (Lab)</i>
<i>Instructor</i>	<i>Mam. Ambreen Gul</i>
<i>Lab Assignment</i>	<i>04</i>

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>FA24-BSE-032</i>	<i>Umair Ahmed</i>

Task 1: Simple RSA Implementation (Without Libraries)

Objective: Understand the math behind RSA

Tasks:

- **Manually implement key generation using small primes**
- **Write functions for: o Encrypting a message (using public key) o Decrypting a message (using private key)**
- **Test it on your name or a short string**

Code:

```

1  # Choosing two prime numbers
2  p = 17
3  q = 19
4
5  # Calculate n and phi
6  n = p * q
7  phi = (p - 1) * (q - 1)
8
9  # Public key exponent (must be coprime with phi)
10 e = 7
11
12
13 # Finding modular inverse (private key d)
14 def mod_inverse(e, phi):
15     for d in range(1, phi):
16         if (e * d) % phi == 1:
17             return d
18     return None
19
20
21 # Private key
22 d = mod_inverse(e, phi)
23
24
25 # Encryption function
26 def encryption(message, e, n):
27     encrypted = []
28
29     for char in message:
30         m = ord(char)      # Convert character to ASCII
31         c = (m ** e) % n   # RSA encryption
32         encrypted.append(c)
33
34     return encrypted

```

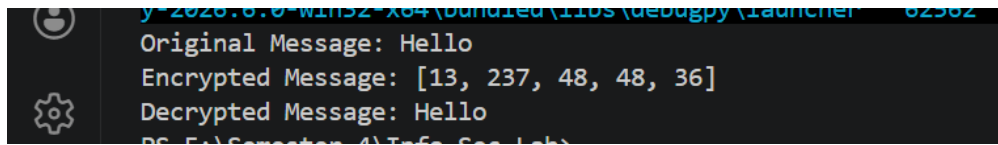
E:\Semester 4 > Info Sec Lab > assignment 4 task 1.py > ...

```

26 def encryption(message, e, n):
31     c = (m ** e) % n      # RSA encryption
32     encrypted.append(c)
33
34     return encrypted
35
36
37 # Decryption function
38 def decryption(cipher, d, n):
39     decrypted = ""
40
41     for c in cipher:
42         m = (c ** d) % n   # RSA decryption
43         decrypted += chr(m) # Convert ASCII back to character
44
45     return decrypted
46
47
48 # Original message
49 message = "Hello"
50
51 print("Original Message:", message)
52
53 # Encrypt
54 cipher_text = encryption(message, e, n)
55 print("Encrypted Message:", cipher_text)
56
57 # Decrypt
58 plain_text = decryption(cipher_text, d, n)
59 print("Decrypted Message:", plain_text)

```

Output:

A terminal window with a dark background and light-colored text. The text shows the output of a program: 'Original Message: Hello', 'Encrypted Message: [13, 237, 48, 48, 36]', and 'Decrypted Message: Hello'. There are some faint, partially visible lines above and below these, including 'y=2020.0.0-win32-x64 (bundled (1105 (debugpy (launcher' and '82582'.

```
y=2020.0.0-win32-x64 (bundled (1105 (debugpy (launcher 82582
Original Message: Hello
Encrypted Message: [13, 237, 48, 48, 36]
Decrypted Message: Hello
PS C:\Program Files\Python\Python38-32>
```

Task 2: Create a Digital Signature

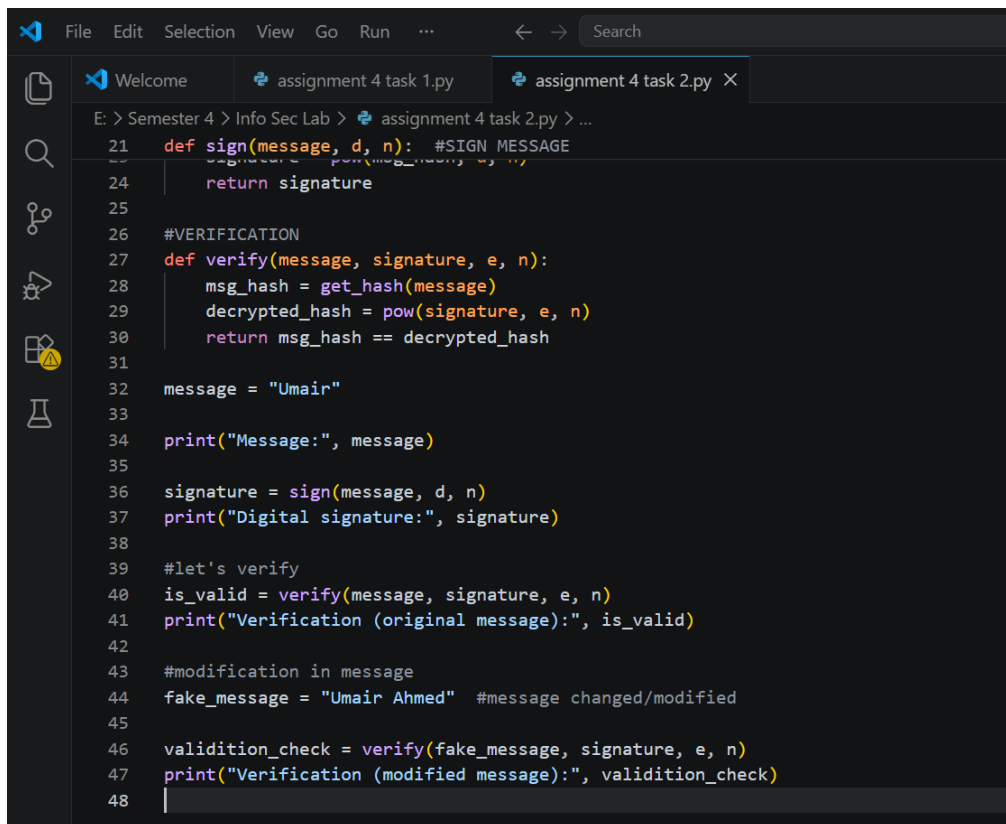
Objective: Understand digital signatures

Tasks:

- Generate RSA key pair
- Create a hash of a message (e.g., using SHA256)
- Sign the hash using private key
- Verify it using the public key
- Modify the message slightly and show that verification fails

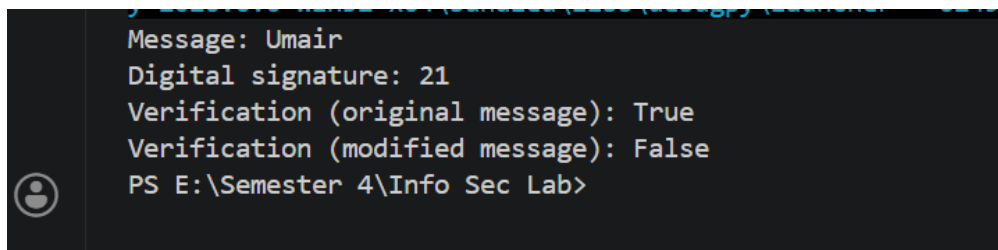
Code:

```
1 import hashlib
2 #key generation
3 p = 5
4 q = 11
5 n = p * q
6 phi = (p - 1) * (q - 1)
7 #public key (must be coprime with phi)
8 e = 3
9
10 def mod_inverse(e, phi): #find private key d
11     for d in range(1, phi):
12         if (e * d) % phi == 1:
13             return d
14     return None
15
16 d = mod_inverse(e, phi)
17
18 def get_hash(message):
19     return int(hashlib.sha256(message.encode()).hexdigest(), 16) % n
20
21 def sign(message, d, n): #SIGN MESSAGE
22     msg_hash = get_hash(message)
23     signature = pow(msg_hash, d, n)
24     return signature
25
26 #VERIFICATION
27 def verify(message, signature, e, n):
28     msg_hash = get_hash(message)
29     decrypted_hash = pow(signature, e, n)
30     return msg_hash == decrypted_hash
31
32 message = "Umain"
33
34 print("Message:", message)
```



```
21 def sign(message, d, n): #SIGN MESSAGE
22     signature = pow(msg_hash, d, n)
23     return signature
24
25
26 #VERIFICATION
27 def verify(message, signature, e, n):
28     msg_hash = get_hash(message)
29     decrypted_hash = pow(signature, e, n)
30     return msg_hash == decrypted_hash
31
32 message = "Umair"
33
34 print("Message:", message)
35
36 signature = sign(message, d, n)
37 print("Digital signature:", signature)
38
39 #let's verify
40 is_valid = verify(message, signature, e, n)
41 print("Verification (original message):", is_valid)
42
43 #modification in message
44 fake_message = "Umair Ahmed" #message changed/modified
45
46 validation_check = verify(fake_message, signature, e, n)
47 print("Verification (modified message):", validation_check)
48 |
```

Output:



```
Message: Umair
Digital signature: 21
Verification (original message): True
Verification (modified message): False
PS E:\Semester 4\Info Sec Lab>
```